

装订线内

不要答题

北京大学信息科学技术学院考试试卷

考试科目：计算机系统导论 姓名：_____ 学号：_____

考试时间：2025 年 12 月 29 日 大班号：_____ 小班号：_____

题号	一	二	三	四	总分
分数					
阅卷人					

北京大学考场纪律

- 1、考生进入考场后，按照监考老师安排隔位就座，将学生证放在桌面上。无学生证者不能参加考试；迟到超过 15 分钟不得入场。在考试开始 30 分钟后方可交卷出场。
 - 2、除必要的文具和主考教师允许的工具书、参考书、计算器以外，其它所有物品（包括空白纸张、手机、或有存储、编程、查询功能的电子用品等）不得带入座位，已经带入考场的必须放在监考人员指定的位置。
 - 3、考试使用的试题、答卷、草稿纸由监考人员统一发放，考试结束时收回，一律不准带出考场。若有试题印制问题请向监考教师提出，不得向其他考生询问。提前答完试卷，应举手示意请监考人员收卷后方可离开；交卷后不得在考场内逗留或在附近高声交谈。未交卷擅自离开考场，不得重新进入考场答卷。考试结束时间到，考生立即停止答卷，在座位上等待监考人员收卷清点后，方可离场。
 - 4、考生要严格遵守考场规则，在规定时间内独立完成答卷。不准交头接耳，不准偷看、夹带、抄袭或者有意让他人抄袭答题内容，不准接传答案或者试卷等。凡有违纪作弊者，一经发现，当场取消其考试资格，并根据《北京大学本科考试工作与学术规范条例》及相关规定严肃处理。
 - 5、考生须确认自己填写的个人信息真实、准确，并承担信息填写错误带来的一切责任与后果。
- 学校倡议所有考生以北京大学学生的荣誉与诚信答卷，共同维护北京大学的学术声誉。

注意事项（答卷前务必仔细阅读!）:

- (1) 本试卷不是按照题目难度排序。题量较大，注意合理安排时间。
- (2) 采用本课程教材和课件的统一前提假设，包括但不限于：x86-64/Linux；System V AMD64 ABI；AT&T 语法；Y86-64；SEQ/PIPE 处理器。现实中有多种解读的，以本课程为准。
- (3) 在答题纸上答题，写明大题和小题的题号。
- (4) 在答题纸上将选择题的回答按行排布，每 10 题换行。
- (5) 不定项选择题每题 3 分，少选得 1 分，错选得 0 分。题目中出现类似“哪些”的描述，不代表一定会有多个答案。

一 不定项选择题（上）（39 分）

1. 观察下面的 C 程序代码:

```
int x = -20;
unsigned u = (unsigned)x;
unsigned v = u + 50u;
```

下列说法哪些正确?

- A. u 的十六进制表示为 0xFFFFFEC
- B. v 的值为 30
- C. 表达式 (x < u) 的值为 1
- D. 表达式 (x == u) 的值为 1
- E. 表达式 (v < u) 的值为 1

2. 对任意 非负 int x, 执行:

```
float f = (float)x;
int y = (int)f;
```

下列说法哪些正确?

(注: float 的有效精度为 24 位; $2^{24}=16777216$)

- A. 若 $0 \leq x \leq 16777216$, 则一定有 $y == x$
- B. 若 $x == 16777217$, 则一定有 $y == 16777216$
- C. 对任意 x, 一定有 $y == x$
- D. 存在某个 x 使得 $y > x$
- E. 若 x 为偶数, 则一定有 $y == x$

3. 对任意 unsigned int x, 定义:

```
unsigned int y = (x >> 5) << 5;
```

下列说法哪些正确?

- A. $y == (x \& \sim 31u)$
- B. $y \leq x$
- C. x - y 的取值一定在 [0, 31]
- D. 若 x 是 32 的倍数, 则 $y == x$
- E. 若 x 不是 32 的倍数, 则 $y == x + 1$

4. 在 x86-64 常见对齐规则下, 给定:

```
struct S {  
    char    c;  
    int     i;  
    short   s;  
    double  d;  
};
```

下列说法哪些正确?

- A. `offsetof(struct S, i) == 4`
- B. `offsetof(struct S, s) == 8`
- C. `offsetof(struct S, d) == 10`
- D. `sizeof(struct S) == 24`
- E. 若改为字段顺序 `double d; int i; short s; char c;`, 则该结构体大小应该会变为 16

5. 观察下面的汇编代码:

```
movq    $0x12345678FFFFFFFF, %rax  
movq    $0x0000000000000001, %rcx  
addl    %ecx, %eax
```

下列哪些说法正确?

- A. 执行后 `%rax == 0x0000000000000000`
- B. 执行后 `%rax == 0x1234567800000000`
- C. 执行后 `CF=1`
- D. 执行后 `ZF=1`
- E. 执行后 `OF=1`

6. 观察下面的汇编代码片段:

```
baz:  
    pushq %rbp  
    movq  %rsp, %rbp  
    subq  $48, %rsp  
    ...  
    callq qux  
    ...  
    leave  
    retq
```

下列说法哪些正确?

- A. `0(%rbp)` 处保存的是调用者的旧 `%rbp`
- B. `8(%rbp)` 处保存的是返回地址
- C. 紧挨着执行 `callq qux` 之前, `%rsp ≡ 8 (mod 16)`
- D. `subq $48, %rsp` 为局部数据预留了 48 字节 (不含保存的 `%rbp` 和返回地址)
- E. 这里的 `leave` 等价于 `addq $48, %rsp; popq %rbp` (假设 `%rbp` 未被改写)

7. 调用一个有 8 个 long 参数的函数 `f(a1, ..., a8)`, 调用点使用如下汇编代码序列 (省略取值细节):

```
movq a1, %rdi
```

```

movq a2, %rsi
movq a3, %rdx
movq a4, %rcx
movq a5, %r8
movq a6, %r9
pushq a8
pushq a7
callq f
addq $16, %rsp

```

下列说法哪些正确？

- A. 在 f 的入口处 (尚未动 %rsp), a7 位于 8(%rsp)
- B. 若把两条 push 顺序改为先 push a7 再 push a8, 则 f 看到的 a7/a8 会互换
- C. 应当由 caller 回收栈上的传参空间 (用 addq \$16,%rsp)
- D. a7,a8 也可以放在 %r10,%r11 中传递
- E. 因为用了栈传参, 所以 f 中必须设置和管理 %rbp 帧指针

8. 在 x86-64 指令集中, 下列哪些指令在指令字节序列中同时包含 regids 字节与 valC 常量字段？

- A. irmovq V, rB
- B. rmmovq rA, D(rB)
- C. call Dest
- D. OPq rA, rB
- E. ret

9. 关于 ret 在 PIPE 中引发的控制冒险, 下列说法哪些正确？

- A. ret 的目标 PC 来自从栈中读出的返回地址, 取指阶段无法提前知道
- B. 只要流水线中存在 ret (如在 D/E/M 任一阶段), 常见策略是设置 F_stall=1 避免取错指令
- C. ret 带来的控制冒险, 可完全靠 forwarding 解决, 不需要 stall
- D. 为了让流水线“继续流动”但不引入错误指令, 常见做法是在 D 阶段注入 bubble
- E. 当 ret 终于在 W 阶段得到 W_valM 后, PC 选择逻辑可用它作为下一条取指地址

10. 关于“局部性 (locality)”, 下列说法哪些正确？

- A. 顺序扫描连续数组通常具有良好的空间局部性
- B. 在短时间内多次访问同一标量变量通常具有良好的时间局部性
- C. 遍历单链表 (指针追踪) 相比遍历连续数组, 通常空间局部性更差
- D. 对行优先 (row-major) 二维数组, 按列遍历通常比按行遍历空间局部性更好
- E. 增大 cache block 大小只会改善时间局部性, 不会影响空间局部性

11. 关于高速缓存的写策略, 下列说法哪些正确？

- A. write-back cache 通常需要每行一个 dirty bit
- B. write-through 设计中常见搭配是 no-write-allocate
- C. write-allocate 在写不命中时不把 block 载入 cache, 直接写到下一层
- D. write-back 的相关控制电路通常比 write-through 简单
- E. write-through 会更快, 因为向下一层的写入是“立刻发生”的

12. 在一个有两级高速缓存 (cache) 的计算机系统里, 设:

- L1 hit time = 1 cycle,
- L1 miss rate = 5%
- L2 hit time = 10 cycles
- L2 local miss rate = 20% (按所有到达 L2 cache 的访问计)
- L2 到内存的 miss penalty = 100 cycles

下列哪些结论正确?

- A. 按所有到达内存的访问计, global miss rate 为 $0.05 \times 0.20 = 1\%$
- B. L1 的平均 miss penalty 为 $10 + 0.20 \times 100 = 30$ cycles
- C. 平均访存时间为 $1 + 0.05 \times 30 = 2.5$ cycles
- D. 若 L2 local miss rate 降到 10%, 平均访存时间会增大
- E. 若 L1 miss rate 降到 2.5%, 平均访存时间为变为 $1 + 0.025 \times 30 = 1.75$ cycles

13. 关于教材里的 memory mountain (吞吐/带宽随工作集大小与 stride 变化) 的解释, 下列说法哪些正确?

- A. 固定 stride 时, 工作集从小到大跨过某级 cache 容量边界, 性能常出现“断崖”
- B. 固定工作集大小时, stride 变大通常降低空间局部性, 从而降低有效吞吐
- C. stride=1 的优势主要来自时间局部性
- D. 该现象同时受 cache 容量与 cache line 大小影响
- E. 只要工作集大小不变, stride 的变化不影响 cache 命中率

二 不定项选择题 (下) (36 分)

14. 三个 C 程序文件, 下面仅观察其中的全局符号 g:

```
a.c: int g = 1;
```

```
b.c: int g;
```

```
main.c: extern int g; int main(){ return g; }
```

关于链接行为, 下列说法哪些正确?

- A. 链接成功, main 中引用的 g 解析到 a.c 的定义
- B. 链接失败, 因为 g 在 a.c 与 b.c 中被“重复定义”
- C. 若把 a.c 改为 `int g;`, 与 b.c 一样, 则链接仍可成功, 最终只分配一个 g
- D. 若把 b.c 改为 `int g = 2;`, 则链接失败
- E. 若把 a.c 中 g 定义改成 `double g = 1.0;`, main.c 中声明 `extern int g;` 并使用, 链接器会报“类型不匹配”错误

15. 关于异常 (exception) 的细分类型, 下列说法哪些正确?

- A. Interrupt 通常由外部设备异步触发, 处理后回到“下一条指令”
- B. Trap 通常在 CPU 内部由执行指令触发, 处理后回到“下一条指令”
- C. Fault 通常表示可以恢复的错误, 处理后回到“下一条指令”
- D. Abort 通常表示不可恢复的错误, 程序一般不会继续执行
- E. 执行访存指令引发的缺页异常, 属于 Trap

16. 关于“进程上下文切换 (context switch)”，下列说法哪些正确？

- A. 进程切换通常发生在内核态，由调度器决定下一个运行的进程
- B. 显式调用 `fork()` 就会发生进程切换
- C. 定时器中断 (timer interrupt) 可能触发调度，从而导致进程切换
- D. 切换进程时需要保存当前进程的寄存器状态，并恢复另一个进程的寄存器状态
- E. 上下文切换不会改变当前进程的虚拟地址空间映射 (页表等)

17. 文件 `t.txt` 的内容恰好是 4 字节: `abcd` (示意)

```
int main(void){
    int fd = open("t.txt", O_RDONLY);
    char x, y;
    fork();
    read(fd, &x, 1);
    read(fd, &y, 1);
    printf("%c%c\n", x, y);
    return 0;
}
```

下列哪些可能作为某一次 `printf` 打印出的那一行内容出现？

- A. `ab` B. `bd` C. `ca` D. `aa` E. `cd`

18. 观察下面的 C 程序代码

```
volatile sig_atomic_t reap = 0;
void handler(int sig){
    while (waitpid(-1, NULL, WNOHANG) > 0)
        reap++;
}
int main(void){
    Signal(SIGCHLD, handler);
    for (int i = 0; i < 2; i++){
        if (fork() == 0) _exit(0);
        while (reap < 2);
        printf("OK\n");
    }
}
```

下列说法哪些正确？

- A. `handler` 可能只被调用 1 次，但 `reap` 最终仍可能变为 2
- B. 程序打印 `OK` 时，通常不会残留 `zombie` 子进程
- C. 因为 `SIGCHLD` 不排队，`reap` 最大只能到 1
- D. 若把 `handler` 里的 `while` 改为只调用一次 `waitpid`，则一定不会留下 `zombie`
- E. `OK` 可能在任意时刻打印 (包括在子进程尚未退出时)

19. 观察下面的 C 程序代码:

```
volatile sig_atomic_t count = 0;
void h(int sig){ count++; }
int main(void){
    sigset_t m;
```

```

sigemptyset(&m);
sigaddset(&m, SIGINT);

Signal(SIGINT, h);
sigprocmask(SIG_BLOCK, &m, NULL);

raise(SIGINT); // raise() 用于向当前进程发出指定信号
raise(SIGINT);

sigprocmask(SIG_UNBLOCK, &m, NULL);
printf("%d\n", (int)count);
}

```

该程序正常执行完，下列说法哪些正确？

- A. count 一定为 0
- B. count 一定为 1
- C. count 可能为 2
- D. 在解除阻塞之前，handler 可能会运行
- E. 在设置阻塞之前，handler 可能会运行

20. 关于 `execve()` 成功执行后的语义，下列说法哪些正确？

- A. 成功的 `execve()` 不会返回到调用点（除非失败）
- B. `execve()` 会用新程序的代码/数据替换当前进程的用户态地址空间
- C. `execve()` 成功后，进程 PID 会改变（变成新程序的 PID）
- D. `execve()` 成功后，默认情况下已打开的文件描述符仍然保持打开（除非设置 `close-on-exec`）
- E. `execve()` 会创建一个新进程，因此父子关系会改变

21. 执行下面的 C 程序代码

```

int *p = malloc(sizeof(int));
*p = 10;
pid_t pid = fork();
if (pid == 0) { *p = 20; _exit(0); }
wait(NULL);
printf("%d\n", *p);

```

下列说法哪些正确？

- A. 输出一定为 20
- B. 输出一定为 10
- C. 输出有可能是 10，也有可能是 20，取决于进程调度顺序
- D. `fork` 之后，父进程由于 Copy-on-Write 可能会发生缺页异常
- E. `fork` 之后，子进程由于 Copy-on-Write 可能会发生缺页异常

22. 关于 TLB，下列说法哪些正确？

- A. TLB 本质上是缓存，缓存最近使用的页表条目
- B. TLB 命中时，CPU 通常无需再访问内存中的页表即可完成地址翻译
- C. 进程上下文切换时 TLB 必须清空，否则会被另一个进程误用

- D. TLB 的 miss rate 由页大小决定, 与程序访问模式无关
- E. 发生 page fault 之前必然有一次对应的 TLB miss

23. 关于动态内存分配, 下列说法哪些正确?

- A. 隐式空闲链表 (implicit list) 通常需要线性扫描找空闲块, 吞吐可能受影响
- B. 显式空闲链表 (explicit free list) 把空闲块串起来, 往往能更快找到候选块, 但空闲块需要额外指针字段
- C. 内部碎片的通常体现: 空闲总量足够但分散, 无法满足较大连续请求
- D. 外部碎片的通常体现: 分配块比请求需求大造成的浪费
- E. 分割 (splitting) 是把大空闲块切成 “已分配块 + 剩余空闲块”, 可能提高利用率但增加管理开销

24. 并发网络服务器的这段程序代码:

```
int listenfd = open_listenfd("8080");
while (1) {
    int connfd = accept(listenfd, NULL, NULL);
    if (fork() == 0) {
        echo(connfd);
        exit(0);
    }
    close(connfd);
}
```

为使该程序更 “正确/健壮”, 下列哪些修改是必要或强烈建议的?

- A. 子进程在 echo(connfd) 前应 close(listenfd)
- B. 父进程在 fork 后应 close(listenfd)
- C. 父进程应处理 SIGCHLD (或循环 waitpid) 以回收子进程, 避免 zombie
- D. 父进程不应 close(connfd), 否则子进程会立即断开
- E. while 循环的判断条件应该加上连接出错时跳出循环

25. 关于进程 (processes) 和线程 (threads) 的描述, 下列说法哪些正确?

- A. 同一进程内的多个线程通常共享同一虚拟地址空间 (代码/全局数据/堆)
- B. 同一进程内的不同线程通常有各自独立的栈 (stack)
- C. 进程切换往往比同进程的线程切换工作量大、耗费时间长
- D. 线程之间共享文件描述符表, 一个线程 close(fd) 可能影响其他线程
- E. 线程之间共享寄存器状态, 一个线程改 %rax 会影响另一个线程

三 链接相关主题 (10 分)

本题基于下列两个 C 语言文件编译生成的 main.o 和 sum.o，编译和运行在 x86-64/Linux 环境下完成，未开启优化选项。

<pre>// main.c #include <stdio.h> extern int sum_array(int *arr, int n); int arr[5] = {1, 2, 3, 4, 5}, result; static int fix = 2010; int main() { result = sum_array(arr, 5)+fix; printf("sum = %d\n", res1); return 0; }</pre>	<pre>// sum.c int sum_array(int *arr, int n) { int sum = 0; for (int i = 0; i < n; i++) { sum += arr[i]; } return sum; }</pre>
--	---

1. 符号表分析 (7 分)

对于下表中给出的每个符号, 用“是”或“否”指出它是否在模块 main.o 的 .symtab 节中有符号表条目。如果存在条目, 请指出符号类型 (局部、全局、外部) 以及定义该符号的模块 (main.o、sum.o、libc.so 或“无”) 和所在节名 (或伪节名)。如果不存在条目, 则填写“/”。

(注: 在答题纸上画出下面完整表格)

	符号	.symtab 有条目?	符号类型	定义符号的模块	节或伪节名
1	arr				
2	result				
3	fix				
4	sum_array				
5	i				
6	sum				
7	printf				

2. 重定位分析 (2 分)

下面是链接后生成的可执行文件 m 使用 objdump -D 反汇编的片段 (部分地址和内容示意)

0000000000001149 <main>:	
1149: 55	push %rbp
114a: 48 89 e5	mov %rsp, %rbp
114d: be 05 00 00 00	mov \$0x5, %esi
1152: bf __ __ __ __	mov \$_____, %edi ; arr 的地址
1157: e8 __ __ __ __	callq _____ ; sum_array

已知以下重定位条目信息。请根据重定位信息填写空白处的机器码 (32 位小端编码)。

(注: 在答题纸上画出下面表格的第 1 列和第 3 列)

编号	重定位条目信息	重定位引用值 32 位编码 (参照代码中的 16 进制形式)
①	r.offset = 0x09 r.symbol = arr r.type = R_X86_64_32 r.addend = 0	
②	r.offset = 0x0e r.symbol = sum_array r.type = R_X86_64_PC32 r.addend = -4	

此处假设 arr 在可执行文件中的地址为 0x402010, sum_array 的地址为 0x1180 result 的地址为 0x404018, main 中 call 指令的下一指令地址为 0x115c

3. 链接顺序影响 (1 分)

执行以下两条命令

①gcc -o m1 main.c sum.c ②gcc -o m2 sum.c main.c

关于生成的两个可执行文件 m1 和 m2, 以下说法正确的是 ()

- A. 两者大小相同, 内容相同 B. 两者大小相同, 内容不同
C. 两者大小不同, 内容相同 D. 两者大小不同, 内容不同

四 并发相关主题 (15 分)

1. (4 分) 以下是一段改编自教材上的多线程代码。对于 thread 函数内出现的以下 8 个变量: i, niters, local_thread, static_local, cnt, index, shared_array, dynamic_ptr。哪几个是共享变量?

答:

```
#define N 100
void *thread(void *vargp);
volatile long cnt = 0;
int shared_array[N];
int *dynamic_ptr;

int main(int argc, char** argv) {
    long niters;
    pthread_t tid1, tid2;
    int local_main = 0;
    if(argc!=2) {
        printf("usage: %s <niters>\n", argv[0]);
        exit(0);
    }
    niters = atoi(argv[1]);
    dynamic_ptr = (int*)malloc(N * sizeof(int));
    if(dynamic_ptr == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    for(int i = 0; i < N; i++) {
        shared_array[i] = i;
    }
    int thread_arg = niters;
    pthread_create(&tid1, NULL, thread, &thread_arg);
    pthread_create(&tid2, NULL, thread, &thread_arg);
    pthread_join(tid1, NULL);
    pthread_join(tid2, NULL);

    if (cnt != 2 * niters) {
        printf("BOOM! cnt = %ld (expected %ld)\n", cnt, 2 * niters);
    } else {
        printf("OK cnt = %ld\n", cnt);
    }
    free(dynamic_ptr);
    exit(0);
}
```

```

}

void* thread(void *arg) {
    long i;
    int niters = *(int *)arg;
    int local_thread = 0;
    static int static_local = 0;
    for(i = 0; i < niters; i++) {
        cnt++;
        int index = i % N;
        shared_array[index]++;
        dynamic_ptr[index]++;
        static_local++;
        local_thread++;
    }
    printf("Thread finished: local_thread=%d, static_local=%d\n",
        local_thread, static_local);
    return NULL;
}

```

2. (3 分) 小明在学习了读者-写者问题后, 编写了以下代码用来解决读者-写者问题。其中, Lightswitch 结构体封装了进入临界区的读者计数 counter 的操作逻辑, 保证当有读者在临界区时, 其他读者仍然可以进入, 但写者无法进入。请阅读下面代码 (代码阅读提示: 主要关注 reader 和 writer 两个函数), 回答问题:

(1) 交换 reader 函数里的 lock(&reader_switch, &wmutex); 和 V(&empty); 两行, 是否会导致竞争或死锁问题? 答: () [是/否]

(2) 在本题的 reader 和 writer 实现基础上, 考虑一共有 3 个读者 (R1, R2, R3) 和 2 个写者 (W1, W2) 依次到来, 且循环仅执行一次 (NUM_OPERATIONS=1) 的情况, 读者执行读操作时间为 7, 写者执行写操作时间为 8, 其到达时间如下: R1:0, R2:1, W1:2, W2:4, R3:5。在时刻 6 时, R1、R2、W1、W2、R3 依次处于的位置可能是:

答: ()

- A. (9), (8), (4), (2), (11)
- B. (11), (8), (3), (3), (8)
- C. (11), (11), (3), (2), (8)
- D. (9), (11), (4), (3), (11)

(注意, 为简单起见, 只考虑读写时间, 忽略所有其他时间, 包括代码中其他语句的执行时间、线程切换、调度等等, 亦假设没有更多的读者或写者或其他线程。)

```

#include "csapp.h"
#define NUM_READERS 5
#define NUM_WRITERS 10
#define NUM_OPERATIONS 100

typedef struct {
    int counter;
    sem_t mutex;
} Lightswitch;

void lightswitch_init(Lightswitch* ls) {
    ls->counter = 0;
}

```

```

        sem_init(&ls->mutex, 0, 1);
    }

void lock(Lightswitch* ls, sem_t* semaphore) {
    P(&ls->mutex);
    ls->counter += 1;
    if (ls->counter == 1) {
        P(semaphore);
    }
    V(&ls->mutex);
}

void unlock(Lightswitch* ls, sem_t* semaphore) {
    P(&ls->mutex);
    ls->counter -= 1;
    if (ls->counter == 0) {
        V(semaphore);
    }
    V(&ls->mutex);
}

void lightswitch_destroy(Lightswitch* ls) {
    sem_destroy(&ls->mutex);
}

sem_t wmutex;
sem_t empty;
Lightswitch reader_switch;

void* writer(void* arg) {
    (1)   for (int i = 0; i < NUM_OPERATIONS; i++) {
    (2)       P(&empty);
    (3)       P(&wmutex);
    (4)       // writing here, need 8 time unit
    (5)       V(&wmutex);
    (6)       V(&empty);
    }
    return NULL;
}

void* reader(void* arg) {
    (7)   for (int i = 0; i < NUM_OPERATIONS; i++) {
    (8)       P(&empty);
    (9)       lock(&reader_switch, &wmutex);
    (10)      V(&empty);
    (11)      // reading here, need 7 time unit
    (12)      unlock(&reader_switch, &wmutex);
    }
    return NULL;
}

int main() {
    pthread_t readers[NUM_READERS];
    pthread_t writers[NUM_WRITERS];
    sem_init(&wmutex, 0, 1);
    sem_init(&empty, 0, 1);
    lightswitch_init(&reader_switch);

```

```

for (int i = 0; i < NUM_READERS; i++)
    pthread_create(&readers[i], NULL, reader, NULL);

for (int i = 0; i < NUM_WRITERS; i++)
    pthread_create(&writers[i], NULL, writer, NULL);

for (int i = 0; i < NUM_READERS; i++)
    pthread_join(readers[i], NULL);

for (int i = 0; i < NUM_WRITERS; i++)
    pthread_join(writers[i], NULL);

sem_destroy(&wmutex);
sem_destroy(&empty);
lightswitch_destroy(&reader_switch);
return 0;
}

```

3. (8 分) 在现代餐饮行业中，高效的订单处理系统至关重要。小红设计了一个模拟在线餐厅的订单处理流程，其中服务员接收顾客订单，厨师处理这些订单。为最大化效率，餐厅雇佣了多个服务员和厨师同时工作，他们通过一个共享的订单缓冲区进行协作。该场景遵循以下规则：

- 服务员有 NUM_WAITERS 名，厨师有 NUM_COOKS 名 (NUM_WAITERS 和 NUM_COOKS 都大于 1)，服务员和厨师使用同一个共享订单缓冲区；
- 每个服务员独立工作，每次将接收到的一个顾客订单放入共享的订单缓冲区内，当缓冲区满时，服务员必须等待，直到有空闲位置才能放入新订单；
- 每个厨师独立工作，每次从共享订单缓冲区内取出一个订单进行烹饪处理，当缓冲区为空时，厨师必须等待，直到有新的订单被放入；
- 订单缓冲区的容量为 BUF_SIZE (BUF_SIZE 大于 0)，用于存放待处理的订单，且初始情况下订单缓冲区为空。

下面代码中，waiter 函数为服务员线程，cook 函数为厨师线程，init_buffer 函数为初始化函数，会在系统启动时调用一次，略去其他部分代码和无关逻辑。

请填写程序中对空白处的代码：

```

#define NUM_WAITERS 4    // 服务员数量
#define NUM_COOKS 2     // 厨师数量
#define BUF_SIZE 5      // 订单缓冲区容量

typedef struct {
    char* orders[BUF_SIZE];
    int in;
    int out;
    sem_t mutex;
    sem_t empty;
    sem_t full;
} Buffer;

Buffer buffer;

void init_buffer() {
    buffer.in = 0;

```

```

    buffer.out = 0;
    sem_init(&buffer.mutex, 0, _____(1)_____);
    sem_init(&buffer.empty, 0, _____(2)_____);
    sem_init(&buffer.full, 0, 0);
}

void* waiter(void* arg) {
    while(1) {
        // 接单逻辑
        char* order = xxxx;
        // 将订单放入缓冲区
        _____(3)_____;
        _____(4)_____;
        buffer.orders[buffer.in] = order;
        buffer.in = (buffer.in+1) % BUF_SIZE;
        V(&buffer.mutex);
        _____(5)_____;
        // 其他逻辑略去
    }
}

void* cook(void* arg) {
    while(1) {
        // 从缓冲区取出订单
        _____(6)_____;
        _____(7)_____;
        char* order = buffer.orders[buffer.out];
        buffer.out = (buffer.out+1) % BUF_SIZE;
        V(&buffer.mutex);
        _____(8)_____;
        // 其他逻辑略去
    }
}

```